

Parallel Computing

Programming with MPI

Mariko & Kevin Dunseath

Computational Science and Applications

2024 – 2025

2024-07-31

Parallel Computing

Parallel Computing
Programming with MPI

Mariko & Kevin Dunseath
Computational Science and Applications

2024 – 2025



- <http://www.open-mpi.org>
- <http://www.idris.fr/formations/mpi>
- <https://mpitutorial.com/>

2024-07-31

Parallel Computing

On-line ressources

Click on text to reach official website

IDRIS: course in French and English, for Fortran and C, reference card, tutorials



- <http://www.open-mpi.org>
- <http://www.idris.fr/formations/mpi>
- <http://mpitutorial.com/>

- API for a library of routines implementing communications between processes

Application **P**rogramming Interface

≡ software connecting computers or computer programs

⇒ tools for building codes that run in parallel

- available in C, C++, Fortran, Python, Java, . . .
- available on many different types of machine
⇒ portable programs
- for distributed memory
- exists on shared memory systems too

2024-07-31

Parallel Computing

└ MPI (Message Passing Interface)

MPI ≡ standard of clearly defined low-level routines upon which vendors can create higher-level routines for the distributed-memory communication environment supplied with their parallel machines.

All the hard, technical details are hidden from the programmer.

MPI provides essential virtual topology, synchronization, and communication functionality between a set of processes (that have been mapped to nodes/servers/computer instances) in a language-independent way.

MPI programs always work with processes, but programmers commonly refer to the processes as processors!

- API for a library of routines implementing communications between processes
- **Application Programming Interface**
 - ≡ software connecting computers or computer programs
 - ⇒ tools for building codes that run in parallel
- available in C, C++, Fortran, Python, Java, . . .
- available on many different types of machine
 - ⇒ portable programs
- for distributed memory
- exists on shared memory systems too

- identify the tasks that can be run in parallel
- insert calls to the MPI library to give instructions to the appropriate processes
- when running the program, decide how many processes to use

Parallelism is the responsibility of the programmer, who must

- identify the tasks that can be run in parallel
- insert calls to the MPI library to give instructions to the appropriate processes
- when running the program, decide how many processes to use

Use Makefile to compile the codes.

Use scripts to run the codes!

Check the number of processors on your computer

- `more /proc/cpuinfo`
- `more /proc/cpuinfo | grep processor`
- `more /proc/cpuinfo | grep 'core id'`

2024-07-31

Parallel Computing

└─ Number of processors

Check the number of processors on your computer

- `more /proc/cpuinfo`
- `more /proc/cpuinfo | grep processor`
- `more /proc/cpuinfo | grep 'core id'`

If the number of core ids is twice smaller than the number of processors $\Rightarrow$ Hyper-Threading Technology $\equiv$ Intel's simultaneous multithreading (SMT) implementation used to improve parallelization of computations (doing multiple tasks at once) performed on microprocessors.

Superscalar architecture: each physical core execute instructions from two different programs during the same clock cycle.

The operating system addresses two virtual (logical) cores and shares the workload between them when possible, increasing the number of independent instructions in the pipeline.

Hyper-threading might not work with MPI, it works with Open-MP although performance is not guaranteed.

A first MPI program

```
#include <stdio>
#include "mpi.h"

int main() {

    int nprocs,rank;

    MPI_Init(NULL,NULL);
    MPI_Comm_size(MPI_COMM_WORLD,&nprocs);
    MPI_Comm_rank(MPI_COMM_WORLD,&rank);

    printf("I am process %d out of %d\n",
           rank,nprocs);
    MPI_Finalize();

    return 0;
}
```

2024-07-31

Parallel Computing

└ A first MPI program

A first MPI program

```
#include <stdio>
#include "mpi.h"

int main() {

    int nprocs,rank;

    MPI_Init(NULL,NULL);
    MPI_Comm_size(MPI_COMM_WORLD,&nprocs);
    MPI_Comm_rank(MPI_COMM_WORLD,&rank);

    printf("I am process %d out of %d\n",
           rank,nprocs);
    MPI_Finalize();

    return 0;
}
```

To compile the program and run it on 2 processors

```
mpic++ -o hello_mpi.exe hello_mpi.cpp  
mpirun -np 2 hello_mpi.exe
```

```
I am process 0 out of 2  
I am process 1 out of 2
```

2024-07-31

└ A first MPI program (2)

Execute the code several times: the order of the processes changes.

Shell script to compile and run the code: compil.sh in MPI1_src.

```
To compile the program and run it on 2 processors  
mpic++ -o hello_mpi.exe hello_mpi.cpp  
mpirun -np 2 hello_mpi.exe  
  
I am process 0 out of 2  
I am process 1 out of 2
```

- Header file: must be included in all programs and functions that make calls to the MPI library

```
#include "mpi.h"
```

2024-07-31

Parallel Computing

└ Basic syntax

To find mpi.h:

```
cd /
```

```
find . -name mpi.h
```

```
⇒ /usr/lib/x86_64-linux-gnu/openmpi/include/mpi.h
```

- Calling MPI functions in C/C++

`MPI_Xxxxx ( . . . . . )`

the first letter after `MPI_` is in capital
the following are in lower case

2024-07-31

└ Basic syntax (2)

- Calling MPI functions in C/C++
`MPI_Xxxxx ( . . . . . )`
the first letter after `MPI_` is in capital
the following are in lower case

- C/C++ are case-sensitive

2024-07-31

Parallel Computing

└ Basic syntax (3)

Basic syntax (3)

- C/C++ are case-sensitive

MPI_Init (NULL, NULL)

Initialization of the MPI execution environment

- Must be called once before any other calls to MPI routines
- Sets the *communicator* for each process
≡ object defining the set of processes which may communicate within the current program

2024-07-31

└ MPI_Init

MPI_Init (NULL, NULL)

Initialization of the MPI execution environment

- Must be called once before any other calls to MPI routines
- Sets the *communicator* for each process
≡ object defining the set of processes which may communicate within the current program

MPI_COMM_WORLD

Communicator including **all** the available processes

≡ network linking each process with all the others

Advanced feature: other communicators can be defined whenever necessary, *e. g.* with fewer processes, for higher performance

2024-07-31

Parallel Computing

└ Default Communicator

For other communicators, see
<https://mpitutorial.com/tutorials/introduction-to-groups-and-communicators/>

MPI_COMM_WORLD
Communicator including **all** the available processes
≡ network linking each process with all the others

Advanced feature: other communicators can be defined whenever necessary, *e. g.* with fewer processes, for higher performance

```
MPI_Comm_size (MPI_COMM_WORLD, &nprocs)
```

Number of processes

For a particular *communicator*, the number of processes involved is obtained and stored in variable `nprocs` using `MPI_Comm_size (communicator, &nprocs)`

2024-07-31

└ MPI_Comm_size

`MPI_Comm_size (MPI_COMM_WORLD, &nprocs)`

Number of processes

For a particular *communicator*, the number of processes involved is obtained and stored in variable `nprocs` using `MPI_Comm_size (communicator, &nprocs)`

`MPI_Comm_rank (MPI_COMM_WORLD, &rank)`

Rank

- For a particular *communicator*, each process is identified by an integer assigned by the system
- Obtained and stored in variable `rank` using `MPI_Comm_rank (communicator, &rank)`
- Ranks are contiguous, starting from 0
- Used to specify conditional instructions
- Used to specify source and destination of messages

2024-07-31

└ MPI_Comm_rank

`MPI_Comm_rank (MPI_COMM_WORLD, &rank)`

Rank

- For a particular *communicator*, each process is identified by an integer assigned by the system
- Obtained and stored in variable `rank` using `MPI_Comm_rank (communicator, &rank)`
- Ranks are contiguous, starting from 0
- Used to specify conditional instructions
- Used to specify source and destination of messages

- terminates the MPI execution environment
- **must be** the last call to an MPI routine in the program

2024-07-31

└ MPI_Finalize()

- terminates the MPI execution environment
- **must be** the last call to an MPI routine in the program

The programmer controls parallelism of data

- decide how to distribute data over processes
- insert appropriate calls to the MPI library

2024-07-31

Parallel Computing

└ MPI (2)

MPI (2)

The programmer controls parallelism of data

- decide how to distribute data over processes
- insert appropriate calls to the MPI library

Data are sent as *messages*

The programmer must

- form the message specifying
 - the size
 - the type, which must be homogeneous
 - the content
- prepare the space where the receiver will collect the message
- specify the sender and the receiver
- call the communication routines to send and to receive the message

Data are sent as *messages*

The programmer must

- form the message specifying
 - the size
 - the type, which must be homogeneous
 - the content
- prepare the space where the receiver will collect the message
- specify the sender and the receiver
- call the communication routines to send and to receive the message

Data are sent as *messages*

A message consists of

- the data to be transferred
- an *envelope* containing
 - 1 the rank of the process receiving the message
 - 2 the rank of the process sending the message
 - 3 a *tag*, which may be used to distinguish messages arriving from the same source but requiring different actions
 - 4 a communicator specifying the group of processes involved

The data to be sent is stored in a contiguous array specified by

- 1 the address of the first element
- 2 the number of elements to be sent
- 3 their type: integers, real numbers, structures, ...

```

#include <stdio>
#include "mpi.h"

int main() {

    int nprocs,rank;
    MPI_Status status;
    MPI_Init(NULL,NULL);
    MPI_Comm_rank(MPI_COMM_WORLD,&rank);
    MPI_Comm_size(MPI_COMM_WORLD,&nprocs);

    int M;
    double x;
    if (rank==0) {
        M=42;
        x=2.5;
        MPI_Send(&M,1,MPI_INT,1,0,MPI_COMM_WORLD);
        MPI_Send(&x,1,MPI_DOUBLE,1,1,MPI_COMM_WORLD);
    }
    else if (rank==1) {
        MPI_Recv(&M,1,MPI_INT,0,0,MPI_COMM_WORLD,&status);
        MPI_Recv(&x,1,MPI_DOUBLE,0,1,MPI_COMM_WORLD,&status);
        M+=1;
        x=0.5*x;
    }
    std::cout << "Process " << rank << " M=" << M << " x=" << x << "\n";

    MPI_Finalize();
    return 0;
}

```

2024-07-31

└ MPI_Send and MPI_Recv

MPI_Send and MPI_Recv

```

// Example: MPI_Send and MPI_Recv
#include <mpi.h>
#include <iostream>
using namespace std;

int main() {
    MPI_Init(NULL, NULL);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &nprocs);

    int M;
    double x;

    if (rank == 0) {
        M = 42;
        x = 2.5;
        MPI_Send(&M, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
        MPI_Send(&x, 1, MPI_DOUBLE, 1, 1, MPI_COMM_WORLD);
    }
    else if (rank == 1) {
        MPI_Recv(&M, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);
        MPI_Recv(&x, 1, MPI_DOUBLE, 0, 1, MPI_COMM_WORLD, &status);
        M += 1;
        x = 0.5 * x;
    }

    cout << "Process " << rank << " M=" << M << " x=" << x << "\n";

    MPI_Finalize();
    return 0;
}

```

```
mpic++ send_recv.cpp  
mpirun -np 2 a.out  
Process 0 M=42 x=2.5  
Process 1 M=43 x=1.25
```

2024-07-31

└ MPI_Send and MPI_Recv (2)

MPI_Send and MPI_Recv (2)

```
mpic++ send_recv.cpp  
mpirun -np 2 a.out  
Process 0 M=42 x=2.5  
Process 1 M=43 x=1.25
```

MPI_Send and MPI_Recv (3)

MPI_Send

`MPI_Send(buffer, len, type, dest, tag, comm)`

MPI_Recv

`MPI_Recv(buffer, len, type, source, tag, comm, status)`

buffer: pointer to or address of the variable or array containing data to be sent or received

len: number of data items to be transferred

type: type of data to be transferred (`MPI_INT`, `MPI_FLOAT`, ...)

destination/source: rank of the receiving or sending process. `MPI_ANY_SOURCE` may be used in a receive operation to listen for messages from any other process

tag: non-negative integer defined by the programmer to label a message. The tags on a send-receive pair must be identical. `MPI_ANY_TAG` may be used to receive a message regardless of its tag

communicator: defines the set of processes involved in the communication, usually `MPI_COMM_WORLD`

status: in a receive operation, it contains the source of the message `status.MPI_SOURCE`, its tag `status.MPI_TAG` and the actual size of the message. It is a structure of type `MPI_Status`.

2024-07-31

Parallel Computing

└ MPI_Send and MPI_Recv (3)



`MPI_Recv(buffer, len, type, source, tag, comm, status)`

`status` is a structure of type `MPI_Status`

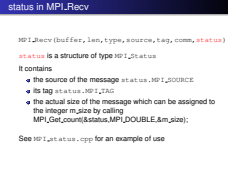
It contains

- the source of the message `status.MPI_SOURCE`
- its tag `status.MPI_TAG`
- the actual size of the message which can be assigned to the integer `m_size` by calling `MPI_Get_count(&status, MPI_DOUBLE, &m_size);`

See `MPI_status.cpp` for an example of use

2024-07-31

└ status in MPI_Recv



MPI defines its own set of data types to ensure portability between systems:

```
MPI_CHAR, MPI_SHORT, MPI_INT, MPI_LONG,  
MPI_UNSIGNED_CHAR, MPI_UNSIGNED_SHORT,  
MPI_UNSIGNED, MPI_UNSIGNED_LONG, MPI_FLOAT,  
MPI_DOUBLE, MPI_LONG_DOUBLE, MPI_BYTE,  
MPI_PACKED
```

Users can also define their own data types

2024-07-31

└ MPI data types

MPI defines its own set of data types to ensure portability between systems:

```
MPI_CHAR, MPI_SHORT, MPI_INT, MPI_LONG,  
MPI_UNSIGNED_CHAR, MPI_UNSIGNED_SHORT,  
MPI_UNSIGNED, MPI_UNSIGNED_LONG, MPI_FLOAT,  
MPI_DOUBLE, MPI_LONG_DOUBLE, MPI_BYTE,  
MPI_PACKED
```

Users can also define their own data types

- 1 Write a code `mpi_IO.cpp` which
 - asks for an integer from the screen
 - prints the process number and the integer
 - 1 Compile the code

```
mpic++ -o mpi_IO.exe mpi_IO.cpp
```
 - 2 Execute the code

```
mpirun -np 2 mpi_IO.exe
```
 - 3 Open another terminal and type in it the command

```
ps -aef | grep mpi_IO.exe
```

How many codes with the same name are running?
 - 4 Type the required data on the screen and analyze the outputs
- 2 Modify the code so that
 - Process 0 asks for another integer from the screen, prints its rank and the new integer
 - The other processes prints their rank and rank + 100

2024-07-31

Tutorial: Screen I/O

Tutorial: Screen I/O

- 1 Write a code `mpi_IO.cpp` which
 - asks for an integer from the screen
 - prints the process number and the integer
- 1 Compile the code

```
mpic++ -o mpi_IO.exe mpi_IO.cpp
```
- 2 Execute the code

```
mpirun -np 2 mpi_IO.exe
```
- 3 Open another terminal and type in it the command

```
ps -aef | grep mpi_IO.exe
```

How many codes with the same name are running?
- 4 Type the required data on the screen and analyze the outputs

- 2 Modify the code so that
 - Process 0 asks for another integer from the screen, prints its rank and the new integer
 - The other processes prints their rank and rank + 100

Write a simple code that

- 1 declares `int i, float f` and `double x`
- 2 Process 0 initialises these variables to some values and prints them
- 3 Process 0 transfers these variables to rank 1
- 4 Process 1 receives these variables and prints them

Modify the code to answer the following questions

- What error message do you get if the variable rather than its address is transferred in `MPI_Send` and `MPI_Recv`?
- What happens if the tag of one `MPI_Send` does not match the tag of the corresponding `MPI_Recv`?
- What happens if the type of the variable in one `MPI_Send` does not match the type of the variable in the corresponding `MPI_Recv`?

2024-07-31

└ Tutorial: MPI_Send and MPI_Recv

Tutorial: MPI_Send and MPI_Recv

Write a simple code that

- declares `int i, float f` and `double x`
- Process 0 initialises these variables to some values and prints them
- Process 0 transfers these variables to rank 1
- Process 1 receives these variables and prints them

Modify the code to answer the following questions

- What error message do you get if the variable rather than its address is transferred in `MPI_Send` and `MPI_Recv`?
- What happens if the tag of one `MPI_Send` does not match the tag of the corresponding `MPI_Recv`?
- What happens if the type of the variable in one `MPI_Send` does not match the type of the variable in the corresponding `MPI_Recv`?

- 1 Modify `prime_serial.cpp` in order to find all the prime numbers up to M by sharing the tasks among 2 processes using MPI
- 2 Modify the code so that process 0
 - asks for and read M from the screen
 - communicates M to processor 1
 - prints the total number of prime numbers up to M

└ Tutorial: Search for prime numbers

- Tutorial: Search for prime numbers
- Modify `prime_serial.cpp` in order to find all the prime numbers up to M by sharing the tasks among 2 processes using MPI
 - Modify the code so that process 0
 - asks for and read M from the screen
 - communicates M to processor 1
 - prints the total number of prime numbers up to M

- 1 Write a program in C++ that exchanges M times an array of N `doubles` between 2 processes. M and N are read in from the screen by processor 0 and communicated to processor 1
- 2 Measure the time used for the exchanges and estimate the bandwidth $\equiv$ number of bits exchanged / time

Use `MPI_Wtime` which returns a double precision value representing the elapsed wall-clock time in seconds for the calling process, measured from some point in the past

```
double t0 = MPI_Wtime()  
double t1 = MPI_Wtime()  
dt=t1-t0
```

- 3 Vary the value of N and present the results in graphical form

2024-07-31

└ Tutorial: Ping-pong

- 1 Write a program in C++ that exchanges M times an array of N `doubles` between 2 processes. M and N are read in from the screen by processor 0 and communicated to processor 1
- 2 Measure the time used for the exchanges and estimate the bandwidth $\equiv$ number of bits exchanged / time
Use `MPI_Wtime` which returns a double precision value representing the elapsed wall-clock time in seconds for the calling process, measured from some point in the past

```
double t0 = MPI_Wtime()  
double t1 = MPI_Wtime()  
dt=t1-t0
```
- 3 Vary the value of N and present the results in graphical form